

Part 4: C Code Implementation Deep Dive

Detailed Walkthrough of Modules, Physics, and Sockets

1. Fixed-Point CAN Protocol Design

Examining ``common/vehicle_protocol.h``: CAN payloads cannot exceed 8 bytes in standard 2.0B frames. To transmit non-integer values (e.g. speed of 55.4 km/h, voltage of 398.2V) without wasting 4-byte IEEE floats, we use fixed-point scaling:

```
#pragma pack(push, 1)
typedef struct {
    uint16_t speed_kph_x10;    // Speed * 10 (0 to 6553.5 km/h) -> 2 bytes
    uint16_t motor_rpm;        // Motor RPM (0 to 15,000 RPM) -> 2 bytes
    int16_t  motor_torque_nm;  // Torque (-500 to +500 Nm) -> 2 bytes
    int8_t   motor_temp_c;     // Motor temp (-40 to +150 °C) -> 1 byte
    uint8_t  gear_state;       // 0: P, 1: R, 2: N, 3: D -> 1 byte
} PcmTelemetryMsg; // Exact total: 8 bytes
#pragma pack(pop)
```

The ``#pragma pack(push, 1)`` directive instructs GCC not to insert padding bytes between struct members, guaranteeing identical memory layout across different compilers and architectures.

2. Dual-Homed I/O Multiplexing in Central Compute

In ``nodes/central_compute.c``, the controller must read from both ``vcan0`` and ``vcan1`` without blocking on either one. It achieves this using POSIX ``select()`` multiplexing:

```
fd_set read_fds;
FD_ZERO(&read_fds);
FD_SET(sock_pt, &read_fds); // vcan0 socket
FD_SET(sock_body, &read_fds); // vcan1 socket

struct timeval tv = { .tv_sec = 0, .tv_usec = 10000 }; // 10ms timeout
int ret = select(max_fd + 1, &read_fds, NULL, NULL, &tv);

if (ret > 0) {
    if (FD_ISSET(sock_pt, &read_fds)) {
        // Handle incoming Powertrain / BMS frame
    }
    if (FD_ISSET(sock_body, &read_fds)) {
        // Handle incoming Body / Cockpit frame
    }
}
```

3. Mathematical Vehicle Physics Simulation

In ``nodes/pcm_node.c``, the vehicle calculates realistic motion dynamics on every loop iteration (``$dt = 20\text{ms}$`):

- **Aerodynamic Drag:** $F_{\text{aero}} = 0.5 \cdot \rho \cdot C_d \cdot A \cdot v^2 \approx 0.0035 \cdot v^2$
- **Net Force:** $F_{\text{net}} = (\tau_{\text{actual}} \cdot r_{\text{gear}}) - (F_{\text{aero}} + F_{\text{roll}})$
- **Acceleration (Newton's 2nd Law):** $a = \frac{F_{\text{net}}}{m_{\text{vehicle}}}$ ($m = 1600 \text{ kg}$)
- **Integration:** $v_{\text{new}} = v_{\text{old}} + a \cdot dt$

4. Battery Energy & Thermal Dynamics

In `nodes/bms_node.c`, electrical current is calculated dynamically from mechanical shaft power:

$$P_{\text{elec}} = \frac{\tau \cdot \omega}{\eta} + P_{\text{parasitic}} \quad \text{implies} \quad I_{\text{pack}} = \frac{P_{\text{elec}}}{V_{\text{pack}}}$$

- **State of Charge (SoC):** Coulomb counting decreases capacity by $\Delta Q = I_{\text{pack}} \cdot dt$.
- **Joule Heating:** Battery internal heating is simulated via $P_{\text{heat}} = I_{\text{pack}}^2 \cdot R_{\text{int}}$, causing cell temperature to rise under hard acceleration.

5. Extensibility: Adding Future Nodes

To add a 6th node (e.g. ADAS radar):

1. Define `CAN_ID_ADAS_TELEMETRY` (e.g. `0x300`) and struct `AdasTelemetryMsg` in `common/vehicle_protocol.h`.
2. Implement `nodes/adas_node.c` using `open_can_socket()` and `can_send_msg()`.
3. Add `adas_node` to the `Makefile` and `scripts/start_sim.sh`.
4. Update `central_compute.c` to consume `0x300` and trigger automatic emergency braking.